

APPLIED MATH 50 LAB 5: GAME THEORY

1. INTRODUCTION

In today's computer lab, we will:

- (1) Show how the ideas of game theory that we discussed in class can be implemented in a differential equation model, and we will discover the analogue of the Nash equilibrium in these models. We will use the models to simulate the examples from class.
- (2) Study the mathematics of the models in detail and show how to calculate the possible Nash equilibria.
- (3) Introduce the *AM 50 competition* based on the unique number game, and show you how to write functions for it.

2. A DIFFERENTIAL EQUATIONS MODEL OF A GAME

In class we discussed game theoretic payoff matrices such as

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix},$$

representing the payoffs of player 1 in a two-player game. The value a_{11} is the payoff if both players choose the first strategy; the value a_{22} is the payoff if both players choose the second strategy. The value a_{12} is the payoff if player 1 chooses the first payoff and player 2 chooses the second. The value a_{21} is the payoff if player 1 chooses the second payoff and player 2 chooses the first. Here, we will consider a symmetric game, where the payoffs for player 2 are given by,

$$A^T = \begin{pmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{pmatrix}.$$

We discussed in class Nash's idea that there can be an equilibrium point of the game, and that in general the equilibrium point is a mixed strategy as opposed to a pure strategy. Here we will show you an explicit implementation of this idea in terms of ordinary differential equations.

Let A be the payoff matrix as discussed above. We will imagine that there are a large number of people (say 1,000) who are all playing the game according to the payoff matrix. Each of them adopts some mixed strategy described by a vector (p, q) , where p is the probability of playing the first strategy, and q is the probability of playing the second strategy. Choosing $p = 1, q = 0$ corresponds to choosing the first strategy, and choosing $p = 0, q = 1$ is the second. Anything in-between is a mixed strategy.¹

¹Note that this framework is slightly more restrictive than that considered in the lecture, since we assume that all players are playing the same strategy.

Now let us suppose that if you choose a strategy with a bigger payoff, you are more likely to choose this strategy in the future, whereas if you choose a strategy with a smaller payoff, you are less likely to choose this strategy. This is a sensible model, in that if you find yourself losing a lot of money playing a game you will change the way that you play it until that doesn't happen anymore.

A simple model that captures this idea is the following set of ordinary differential equations. These equations are called the “replicator dynamics,” and were introduced by Taylor and Jonker.² The model is

$$(1) \quad \frac{dp}{dt} = p(\text{Payoff}_1 - \overline{\text{Payoff}}),$$

$$(2) \quad \frac{dq}{dt} = q(\text{Payoff}_2 - \overline{\text{Payoff}}),$$

where

$$\text{Payoff}_1 = pa_{11} + qa_{12}$$

and

$$\text{Payoff}_2 = pa_{21} + qa_{22}.$$

These are the payoffs expected from playing each of the two strategies. The quantity $\overline{\text{Payoff}}$ is the *average payoff*. This is given by

$$\overline{\text{Payoff}} = p\text{Payoff}_1 + q\text{Payoff}_2.$$

The model therefore says that if the payoff of a particular strategy is larger than the average payoff, the probability of playing this strategy will grow, whereas if the payoff is less than the average the probability of playing this strategy will shrink. The question we need to consider is whether these dynamics will lead to an equilibrium, and if it agrees with the Nash equilibrium discussed in class.

Exercise 1

Start with the Matlab program we gave you for solving the SIR model from Lab 1. Modify these equations to solve the replicator dynamics described above. Note that in Eqs. 1 & 2 we have that $p + q = 1$, because the total probability of choosing one strategy or the other must sum to unity. Thus when you choose initial conditions, you need to choose them so that the initial $p + q = 1$.

To simulate the equation properly, you need to choose different examples for the matrix A . We propose you start with the three examples we discussed in class.

²There is a very nice description of these dynamics in Martin Nowak's book “Evolutionary dynamics,” and in his course in the math department.

2.1. **The Prisoner's Dilemma.** Recall for the prisoner's dilemma we had the payoff matrix

$$A = \begin{pmatrix} -1 & -10 \\ 0 & -7 \end{pmatrix}.$$

Exercise 2

Simulate the replicator dynamics for the Prisoner's Dilemma. What happens? Start out with several sets of initial conditions (*i.e.* $(p, q) = (1, 0), (0.2, 0.8), (0, 1)$). Does it agree with our Nash equilibrium from class?

2.2. **Snowdrift Game.** Recall the snowdrift game example from class,³

$$A = \begin{pmatrix} R & S \\ T & P \end{pmatrix}$$

with $T > R > S > P$.

Exercise 3

Simulate the replicator dynamics with $T = 4, R = 3, S = 2, P = 1$. What happens? Start out with several sets of initial conditions (*i.e.* $(p, q) = (1, 0), (0.2, 0.8), (0, 1)$). Does it agree with our result from class?

3. ANALYZING THESE RESULTS

We now want to understand more generally the results we found above. Looking back at Eqs. 1 & 2 we can see that they are redundant, since we know that $p + q = 1$. If we take the equation for p and substitute $q = 1 - p$ we get just a single equation for p ,

$$(3) \quad \frac{dp}{dt} = p(1 - p) (\text{Payoff}_1 - \text{Payoff}_2).$$

In terms of our general payoff matrix, this is just

$$(4) \quad \frac{dp}{dt} = p(1 - p) ((a_{11} - a_{12} - a_{21} + a_{22})p + a_{12} - a_{22}).$$

Now, the *equilibria* of the replicator dynamics are values of p where

$$\frac{dp}{dt} = 0.$$

Exercise 4

Find the values of p which can be equilibria. Can you use this to rationalize the results from the simulations above? (*Note that there will be three possible equilibria, but only two of them always satisfy $0 \leq p \leq 1$, as is required by the analysis.*)

³Within the current framework where all players play the same strategy, this example has no pure Nash equilibria. There is a Nash equilibrium where one player plays the first strategy and the other plays the second strategy (or *vice versa*) but this is not considered in the replicator dynamics framework.

4. THE AM 50 COMPETITION: THE UNIQUE NUMBER GAME

4.1. Introduction. In the unique number game, there are N players and each must choose an integer between 0 and 99. The winner is the person who chose the smallest number that was not chosen by anyone else. For example, if $N = 5$ the players might make the following choices:

Player	A	B	C	D	E
Choice	0	5	2	3	0

In this example, players A and E both chose zero so they invalidate each other. Player C made the lowest remaining choice and is therefore the winner. In certain rare cases, such as if B and D switched their choices to two, a game may result in no winner.

If the unique number game is played with human players, then choosing a number requires some psychology about what the other players will pick. Lower numbers are more desirable, but they are risky choices since there may be a greater probability that others will also choose them.

For many repeated games, we expect that a mixed-strategy Nash equilibrium may exist, where different numbers are chosen with different probabilities.

4.2. The game. We are going to play many repeated unique number games in AM 50. Your job is to write a Matlab function that implements a strategy in this repeated game. The function should be called `firstname_lastname.m` and should accept five inputs N , `history`, `lw`, `lc`, `y`. These inputs will be discussed in more detail later. A simple strategy, given below, would be just to pick a fixed number:

```
function r = simple_strategy(N, history, lw, lc, y)

% Always return 10
r = 10;

end
```

Alternatively, one could choose the numbers from 0 to 25 with equal probability:

```
function r = random_strategy(N, history, lw, lc, y)

%Return a random integer between 0 and 25
r = randi([0,25]);

end
```

Your function `firstname_lastname.m` should be submitted as part of the homework (due on Friday April 21st).

Once we have collected all of the programs we will run (hopefully) one hundred sets of ten million rounds each. The number of wins for each player will be counted across all 10^9 rounds. The winner will be rewarded with an amazing prize!

4.3. **Testing a strategy.** We have provided you with a program called `game_test.m` that can be used to test different strategies against each other. It is currently set up to simulate 10,000 rounds with $N = 25$ players. 25 of the players use the `random_strategy.m` strategy, and the last player uses a very basic strategy in `simple_strategy.m`. Currently the player using the simple strategy only wins a small fraction of approximately 0.38% of the rounds, much less than an equal share of $\frac{1}{25} = 4\%$ of the rounds.

Exercise 5

Modify `simple_strategy.m` so that it wins consistently more than 6% of the rounds.

4.4. **More available information.** As in the two sample functions above, Your function `firstname_lastname.m` will be passed additional information about the current state of the game and what has happened in previous rounds. If you wish, you can make use of some of this information to devise a strategy. The five inputs are as follows:

`N` The total number of players.

`history` A list of length 100, containing the total number of choices of each number across all previous rounds that have been played so far.

`lw` The winning number in the previous round. If there was no winner in the previous round this will be set to 100. In the very first round, before anything has been played, `lw` will be set to zero.

`lc` A list of length `N` of all of the players' choices in the previous round. In the very first round, before anything has been played, all entries will be set to zero. Each player will occupy the same entry in the list throughout the game; for example, by repeatedly examining `lc[5]` you could see the entire history of one particular player.

`y` Your position within the `lc` list. Hence `lc[y]` will be equal to your choice in the previous round.

Two functions that make use of some of this information are provided for you to review:

`prev.m` If the previous winning choice was `lw`, then that may be a good number in general. This function returns `lw-1`, `lw`, or `lw+1` with equal probability.

`histo.m` This function chooses the lowest number whose probability of occurrence in the previous games is less than $\frac{1}{N}$, and adds a small random displacement.

The homework will have some additional exercises that may give you pointers on devising a good strategy. After this, you can do the following:

Homework Question 1

Devise a strategy for the unique number game following the description and rules above. Anything is acceptable, and you can even copy verbatim or modify one of the basic examples discussed above. Submit it with the game theory homework, with name `firstname_lastname.m`.

4.5. **Technicalities.** The following technicalities should not be relevant for any basic strategy but may come into play if you want to try something more elaborate:

- (1) We will review all submissions prior to playing the game. If we notice any functions that are liable to crash, we may contact you or make minor fixes.
- (2) Designing strategy that involves cooperation beforehand with other students is allowed. For example, if you restrict your choice to odd numbers, and your friend restricts their choice to even numbers, then at least you know that you will never invalidate each other's choices. However, this is a small advantage that may be outweighed by many other factors. Within the game, active communication between two strategies, such as by sharing data, is not allowed. The positions of each player within `lc` will be randomized, so there will be no straightforward way to identify your friend.
- (3) If your function returns a value less than zero, it will be treated as choosing zero. If your function returns a value greater than 99, it will be treated as choosing 99.
- (4) There will be a limit on how long your function can take. If a function (a) takes longer than 0.1 s more than 100 times in any set of rounds, (b) ever takes longer than 10 s, or (c) crashes, it will be disqualified and the game will be reset to not include this function. The Matlab functions `tic` and `toc` can help you determine how long it takes your function to run.
- (5) The teaching staff may disqualify any entry that is deemed to be subverting the rules or not in the spirit of the game.